Department of Computer Science and Engineering
Premier University

CSE 454: Compiler Construction Laboratory

Lab Report 03: Write a program to identify instances of unique keywords from a C program

Submitted by:

Name	Mohammad Hafizur Rahman Sakib
ID	0222210005101118
Section	C
Session	Fall 2025 (8th Semester)
Date of performance	31.01.2026
Submission Date	03.02.2026

Submitted to:
Nazma Akther
Assistant Professor, Department of CSE
Premier University,Chattogram

Remarks

Experiment No 03 :

Experiment Name :

Write a program to identify instances of unique keywords from a C program.

Objective :

The objective of this experiment is to design and implement a C program that scans a given C source file, identifies all unique C language keywords, and displays their occurrences. This experiment helps in understanding lexical analysis, keyword recognition, and file handling concepts used in compiler design.

Algorithm :

1. Open the input C file in read mode.
2. Read the file character by character.
3. Extract words containing only alphabetic characters.
4. Compare each word with the list of C keywords.
5. Store unique keywords and count their occurrences.
6. Display the identified keywords with their counts.
7. Close the file.

Source Code :

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX_KEYWORDS 32
#define MAX_WORD_LEN 20

struct keyword {
    char word[MAX_WORD_LEN];
    int count;
};

int isKeyword(char word[]) {
    char *keywords[MAX_KEYWORDS] = {
        "auto","break","case","char","const","continue","default",
        "do","double","else","enum","extern","float","for","goto",
        "if","int","long","register","return","short","signed",
        "sizeof","static","struct","switch","typedef","union",
        "unsigned","void","volatile","while"
    };

    for (int i = 0; i < MAX_KEYWORDS; i++) {
        if (strcmp(word, keywords[i]) == 0)
            return 1;
    }
    return 0;
}

int findKeyword(struct keyword k[], int count, char word[]) {
    for (int i = 0; i < count; i++) {
        if (strcmp(k[i].word, word) == 0)
            return i;
    }
    return -1;
}

int main() {
    FILE *fp;
    char ch, word[MAX_WORD_LEN];
    struct keyword found[MAX_KEYWORDS];
    int i = 0, kcount = 0;

    fp = fopen("input.c", "r");
    if (fp == NULL) {
        printf("Error: File not found!\n");
        return 0;
    }

    while ((ch = fgetc(fp)) != EOF) {
        if (isalpha(ch)) {
            i = 0;
            while (isalpha(ch)) {
                word[i++] = ch;
                ch = fgetc(fp);
            }
            word[i] = '\0';

            if (isKeyword(word)) {
                int index = findKeyword(found, kcount, word);
                if (index == -1 && kcount < MAX_KEYWORDS) {
                    strcpy(found[kcount].word, word);
                    found[kcount].count = 1;
                    kcount++;
                } else if (index != -1) {
                    found[index].count++;
                }
            }
        }
    }

    fclose(fp);

    printf("\nKeyword\t\tCount\n");
    printf("----------------------\n");
    for (i = 0; i < kcount; i++) {
        printf("%-10s\t%d\n", found[i].word, found[i].count);
    }

    return 0;
}

```

Figure 1: Source code of the keyword identification program

Input :

The input file `input.c` contains the following C program.

```c
#include <stdio.h>

int main()
{
    float celsius, fahrenheit;

    printf("Enter temperature in Celsius: ");
    scanf("%f", &celsius);

    fahrenheit = (celsius * 9 / 5) + 32;

    printf("%.2f Celsius = %.2f Fahrenheit", celsius, fahrenheit);

    return 0;
}

```

Figure 2: Input code of the keyword identification program

Output Screenshot :

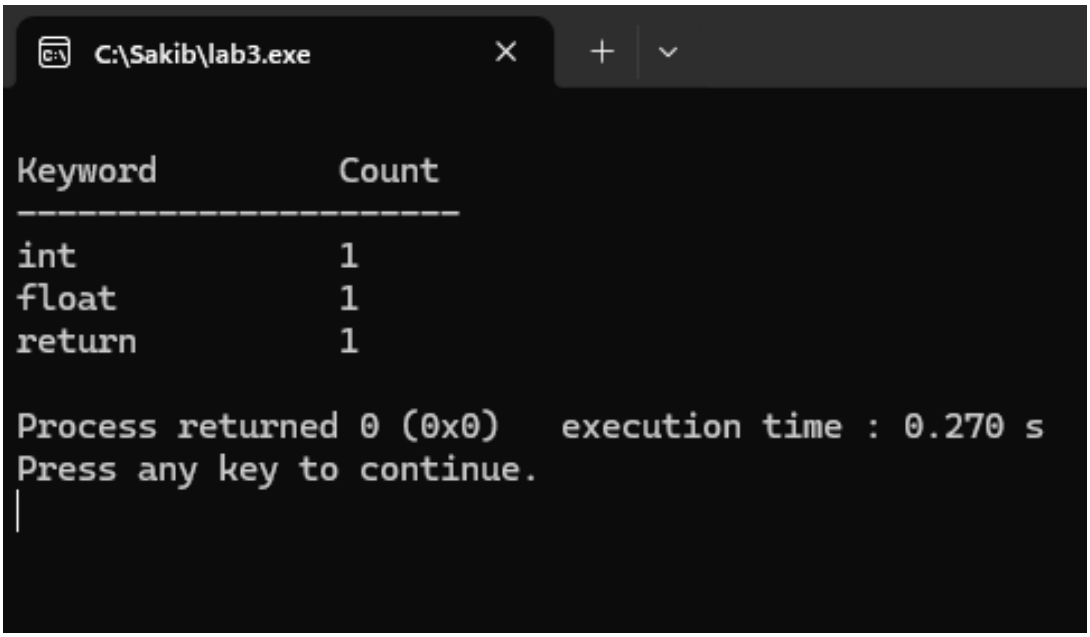

Figure 3: Output of the keyword identification program

Discussion

This experiment demonstrates the identification of unique C keywords using a simple lexical analysis approach. The program successfully scans the source file, recognizes valid keywords, and counts their occurrences. The experiment highlights important compiler design concepts such as tokenization, keyword matching, and file processing.